

Guía de Laboratorio S04 Sistemas NewSQL Big Data

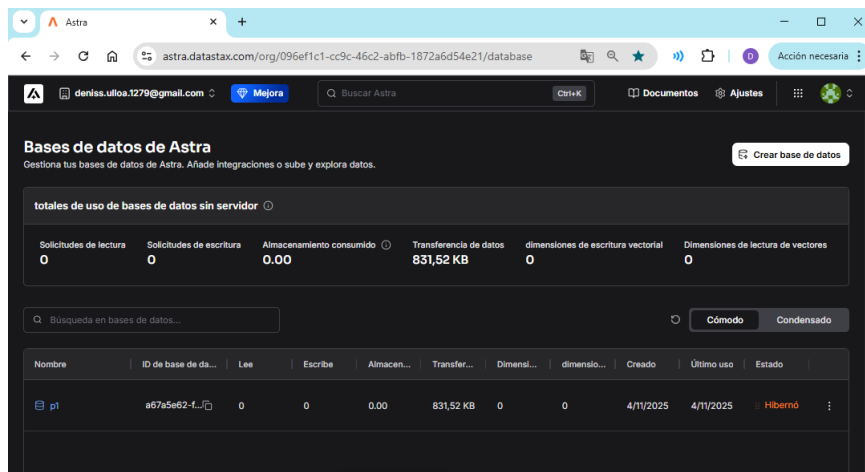
Usar **Cassandra sin instalar nada con DataStax Astra DB (Serverless).**

Paso a paso (desde cero)

Objetivo del laboratorio: aplicar CQL básico, claves de partición, clustering, TTL, colecciones, consistencia, LWT, contadores y modelado orientado a consultas en Cassandra usando Astra DB. La guía actual ya trabaja Astra DB, CQL Console, keyspace `iot`, tablas `personas`, `sensor_data`, colecciones, LWT, índices, TTL, counters y time-series

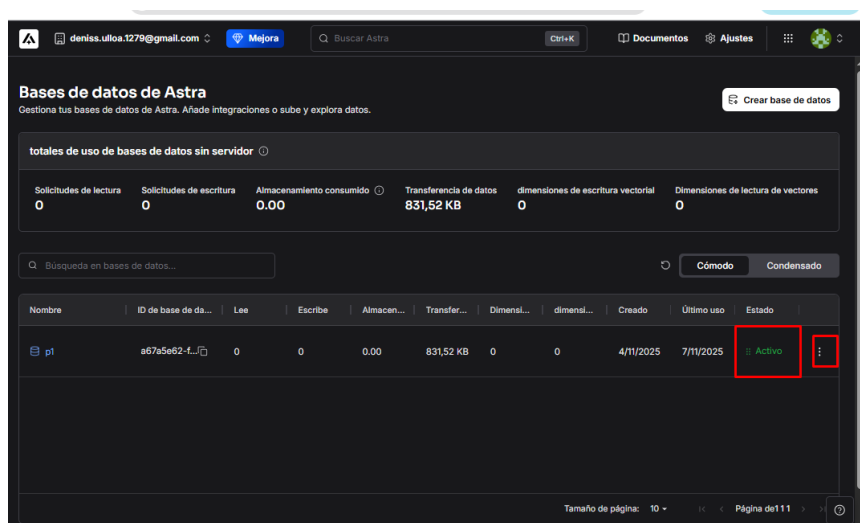
1) Crear cuenta (gratis)

1. Entra a **astra.datastax.com**.
2. Click **Sign up** (puedes usar Google/GitHub o correo).
3. Verifica tu correo y entra.



2) Crear tu base de datos serverless

1. Botón **Create Database**.



2. Tipo: **Serverless**. (sin vector)

3. Completa:
 - **Database name:** por ejemplo cass-semana13
 - **Keyspace:** por ejemplo iot (minúsculas, sin espacios)
 - **Region:** la más cercana a ti (p. ej. us-east-1 o similar)
4. Click **Create**. Espera hasta que el estado sea **Active**.

The screenshot shows the 'Create database' interface in the Astra console. The 'Serverless (non-vector)' option is selected and highlighted with a red box. Below it, the 'Database name' field contains 'cass-semana13', the 'Keyspace name' field contains 'iot', the 'Provider' is set to 'Google Cloud', and the 'Region' is set to 'us-east1'. All these fields are also highlighted with red boxes. The 'Create database' button is visible at the bottom right.

Create database

Serverless (vector)
An all-in-one database solution, optimized for Vector and Generative AI workloads

Serverless (non-vector) ☒
A more traditional database solution without any of our new vector capabilities

Database name *
cass-semana13

Give it a memorable name – this can't be changed later.

Keyspace name * ⓘ
iot

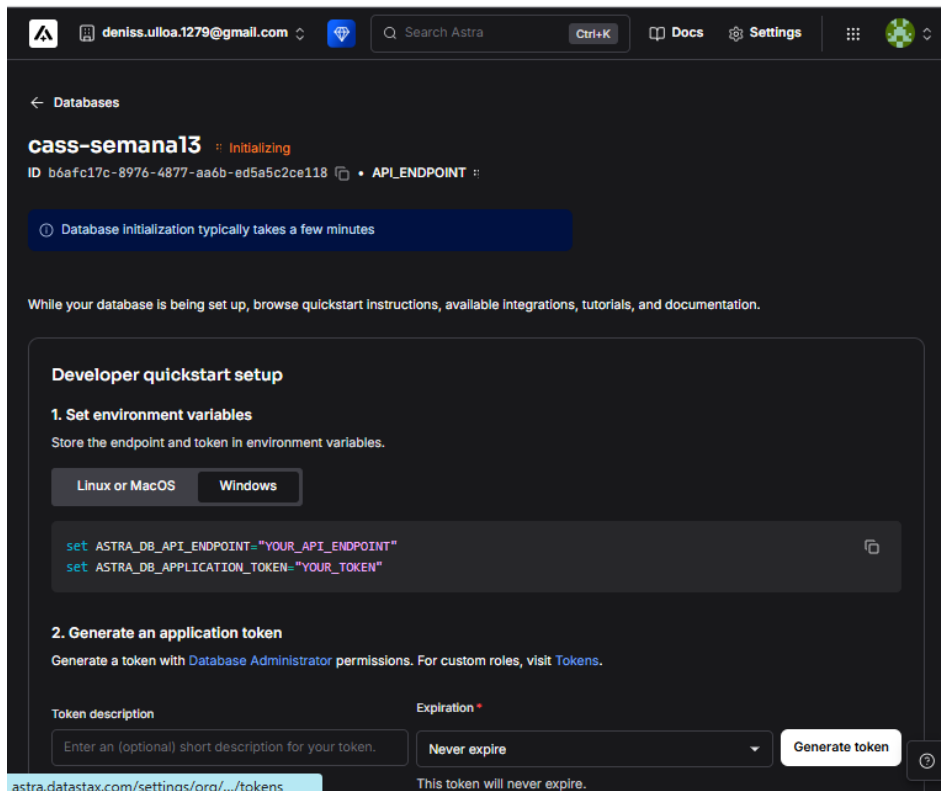
Learn more about [keyspaces](#) and how to use them.

Provider *
☒ Google Cloud ☐ Amazon Web Services ☐ Microsoft Azure

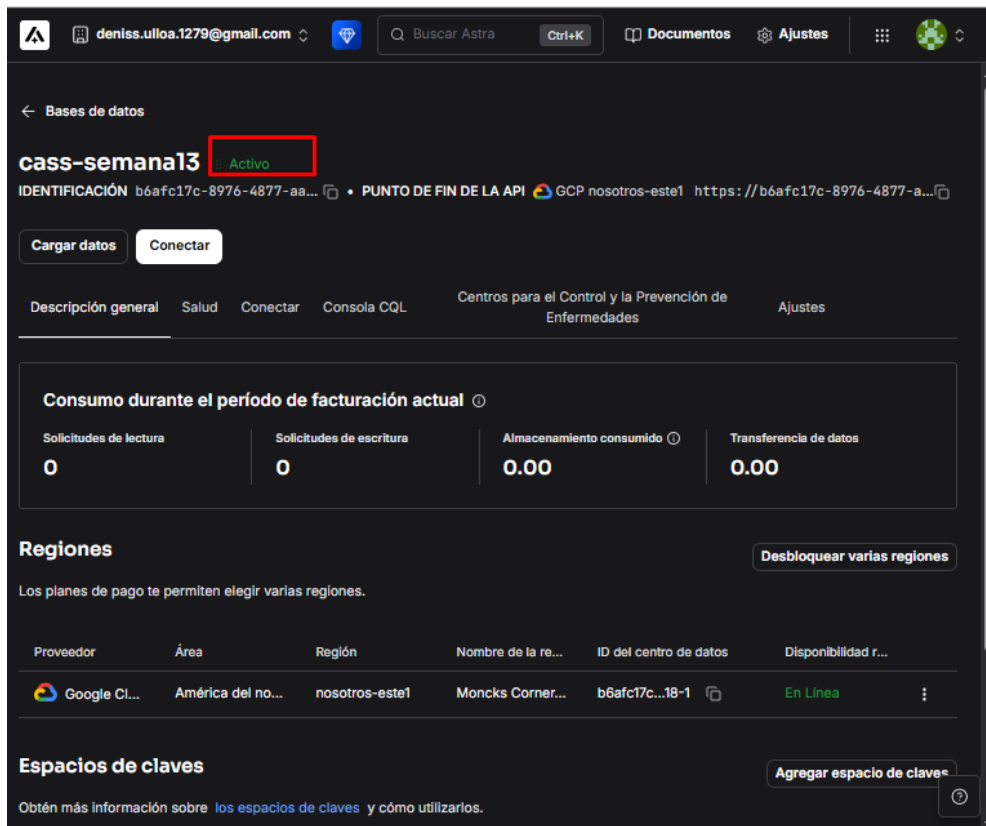
Region *
us-east1

Cancel **Create database**

Esto no instala nada en tu PC. Es un Cassandra administrado en la nube.

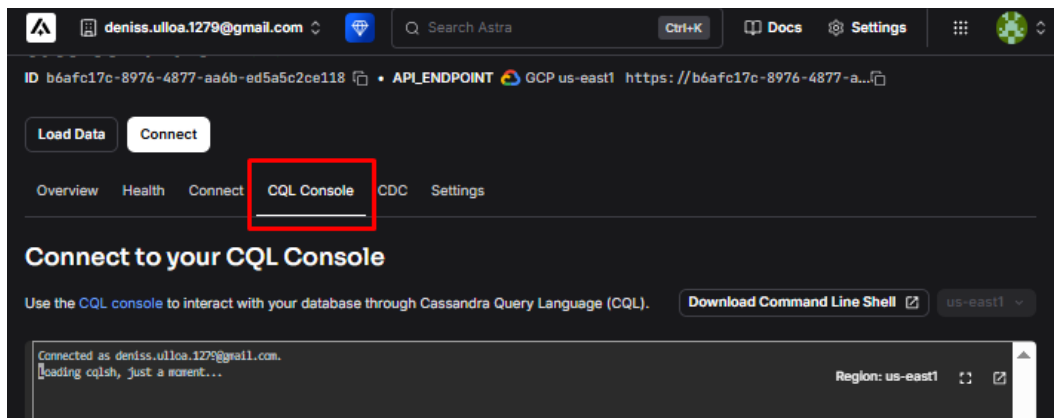


Se espera que el estado pase de *Initializing* → *Active*



3) Abrir la CQL Console en el navegador

1. Entra a tu DB recién creada.
2. En el panel, abre **CQL Console** (o desde **Astra DB Studio** → **CQL Console**).
3. Ya estás conectado a tu clúster Cassandra.



Se realiza la primera tabla y consultas desde la Consola CQL del navegador.

Paso 1: Abrir la Consola CQL

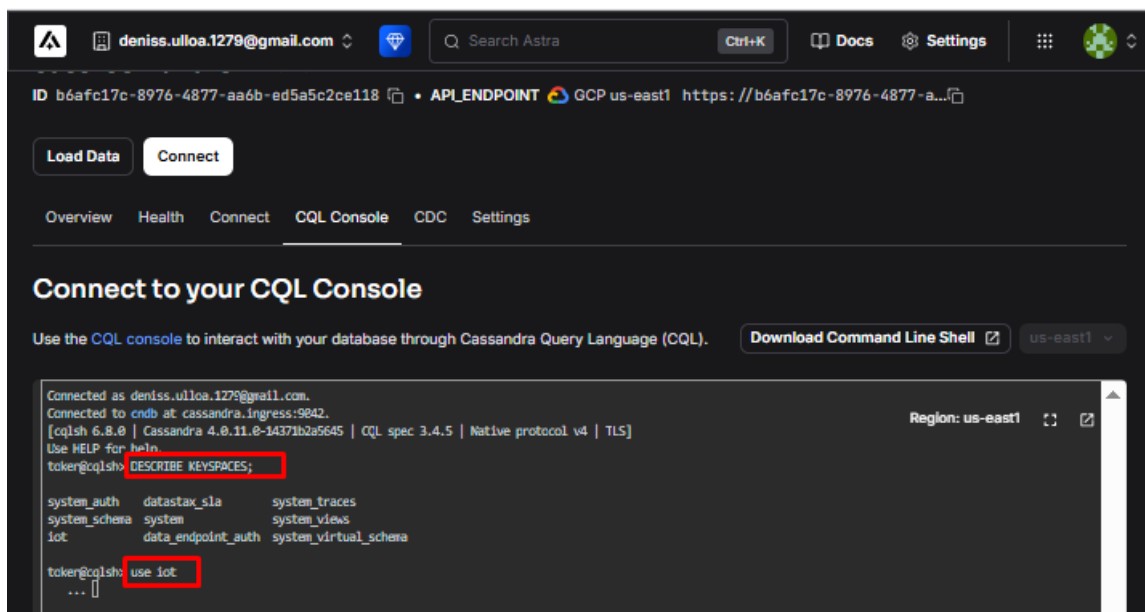
1. En esa misma página, entra a la pestaña **Consola CQL**
2. Se abrirá un editor con un prompt para ejecutar comandos CQL.

Paso 2: Confirmar y usar tu keyspace

DESCRIBE KEYSPACES;

Ubica el tuyo (sale como `iot` en minúsculas normalmente).
Luego cámbiate:

USE `iot`; -- si tu keyspace aparece como `iot`
-- si te aparece exactamente `IoT`, igual usa: USE `iot`;



Paso 3: Crear tu primera tabla

```
CREATE TABLE IF NOT EXISTS tabla_first (  
  id text,  
  nombre text,  
  edad int,  
  ciudad text,  
  PRIMARY KEY (id) -- id será la clave de partición  
);
```

Paso 4: Insertar datos de prueba

```
INSERT INTO tabla_first (id, nombre, edad, ciudad)  
VALUES ('1','Juan',23,'Lima');
```

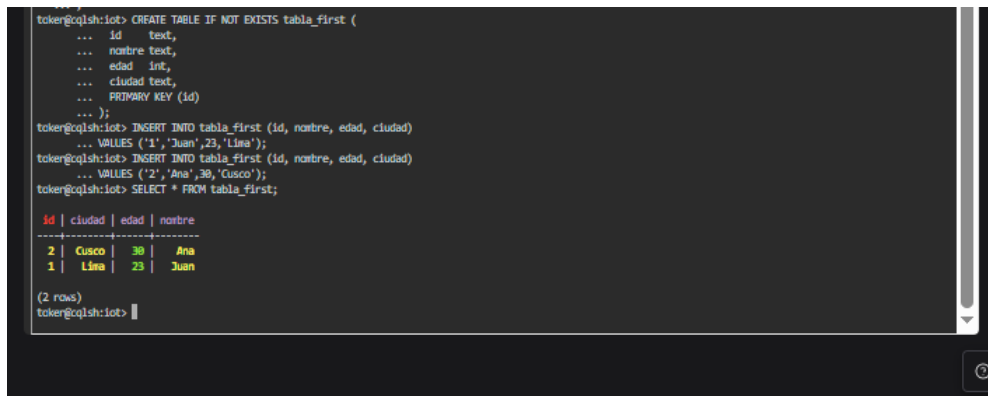
```
INSERT INTO tabla_first (id, nombre, edad, ciudad)  
VALUES ('2','Ana',30,'Cusco');
```

Paso 5: Consultar

```
SELECT * FROM tabla_first;
```

-- Ejemplo de consulta puntual:

```
SELECT nombre, ciudad FROM tabla_first WHERE id = '1';
```



The screenshot shows a terminal window with a dark background. It displays the execution of several SQL commands in a PostgreSQL-like environment. The commands include creating a table, inserting two rows of data, and querying the table. The output shows a table with two rows: one for 'Ana' in 'Cusco' and one for 'Juan' in 'Lima'. The terminal prompt is 'token@qqlsh:iot>'.

```
token@qqlsh:iot> CREATE TABLE IF NOT EXISTS tabla_first (  
... id text,  
... nombre text,  
... edad int,  
... ciudad text,  
... PRIMARY KEY (id)  
... );  
token@qqlsh:iot> INSERT INTO tabla_first (id, nombre, edad, ciudad)  
... VALUES ('1','Juan',23,'Lima');  
token@qqlsh:iot> INSERT INTO tabla_first (id, nombre, edad, ciudad)  
... VALUES ('2','Ana',30,'Cusco');  
token@qqlsh:iot> SELECT * FROM tabla_first;  
  
 id | ciudad | edad | nombre  
----+-----+-----+-----  
  2 | Cusco  |  30  | Ana  
  1 | Lima   |  23  | Juan  
  
(2 rows)  
token@qqlsh:iot> |
```

```
CREATE TABLE personas_ordenadas (  
  ciudad text,  
  id int,  
  nombre text,  
  PRIMARY KEY ((ciudad), id)  
);
```

```
INSERT INTO personas_ordenadas (ciudad,id,nombre)  
VALUES ('Lima',1,'Juan');  
INSERT INTO personas_ordenadas (ciudad,id,nombre)  
VALUES ('Lima',2,'Ana');  
INSERT INTO personas_ordenadas (ciudad,id,nombre)  
VALUES ('Lima',3,'Pedro');
```

```
INSERT INTO personas_ordenadas (ciudad,id,nombre)  
VALUES ('Cusco',1,'Carlos');  
INSERT INTO personas_ordenadas (ciudad,id,nombre)  
VALUES ('Cusco',2,'Maria');
```

```
SELECT * FROM personas_ordenadas;
```

A) Tabla personas (CRUD, TTL, ALTER, BATCH)

Keyspace usado: iot

Primero: USE iot;

A1. Crear tabla

```
CREATE TABLE IF NOT EXISTS personas (  
  id text,  
  nombre text,  
  edad int,  
  ciudad text,  
  email text,  
  PRIMARY KEY (id)  
);
```

Verificación

```
DESCRIBE TABLE personas;
```

Salida esperada (esquema resumido)

columna	tipo	clave
id	text	PRIMARY KEY (partition)
nombre	text	
edad	int	
ciudad	text	
email	text	

A2. Inserts (uno con TTL)

```
INSERT INTO personas (id,nombre,edad,ciudad,email)  
VALUES ('1','Juan',23,'Lima','juan@demo.com');
```

```
INSERT INTO personas (id,nombre,edad,ciudad,email)  
VALUES ('2','Ana',30,'Cusco','ana@demo.com') USING TTL 3600;
```

```
INSERT INTO personas (id,nombre,edad,ciudad,email)  
VALUES ('3','Luisa',28,'Arequipa','luisa@demo.com');
```

```
INSERT INTO personas (id,nombre,edad,ciudad,email)  
VALUES ('4','Pedro',35,'Lima','pedro@demo.com');
```

```
INSERT INTO personas (id,nombre,edad,ciudad,email)  
VALUES ('5','Carlos',29,'Trujillo','carlos@demo.com');
```

```
INSERT INTO personas (id,nombre,edad,ciudad,email)  
VALUES ('6','María',27,'Cusco','maria@demo.com');
```

Verificación

```
SELECT id,nombre,edad,ciudad,email, TTL(email) AS ttl_email  
FROM personas;
```

Salida esperada (ejemplo)

id	nombre	edad	ciudad	email	ttl_email
1	Juan	23	Lima	<u>juan@demo.com</u>	null
2	Ana	30	Cusco	<u>ana@demo.com</u>	3xxx
3	Luisa	28	Arequipa	<u>luisa@demo.com</u>	null

(ttl_email ≈ segundos restantes; solo en la fila con TTL.)

-- Ver cuánto tiempo le queda al dato con TTL

```
SELECT id, nombre, email, TTL(email) AS tiempo_restante
FROM personas
WHERE id='2';
```

-- Insertar otro registro con TTL corto

```
INSERT INTO personas (id,nombre,edad,ciudad,email)
VALUES ('7','Rosa',24,'Lima','rosa@demo.com') USING TTL 60;
```

```
SELECT id,nombre,email,TTL(email) AS ttl_email
FROM personas
WHERE id='7';
```

A3. Leer / Actualizar / Borrar

```
SELECT * FROM personas WHERE id='1';
UPDATE personas SET ciudad='Arequipa' WHERE id='1';
DELETE FROM personas WHERE id='3';
SELECT id,nombre,ciudad FROM personas;
```

Salida esperada (tras update+delete)

id	nombre	ciudad
1	Juan	Arequipa
2	Ana	Cusco

A4. ALTER + BATCH

```
ALTER TABLE personas ADD telefono text;
ALTER TABLE personas ADD estado text;
```

```
BEGIN BATCH
UPDATE personas SET telefono='+51-999111222', estado='Activo' WHERE id='1';
UPDATE personas SET telefono='+51-999222333', estado='Activo' WHERE id='2';
UPDATE personas SET telefono='+51-999333444', estado='Inactivo' WHERE id='4';
APPLY BATCH;
```

```
SELECT id,nombre,ciudad,telefono,estado
FROM personas;
```

Salida esperada

id	nombre	telefono
1	Juan	+51-999111222
2	Ana	+51-999222333

-- Consultar solo personas activas

```
SELECT id,nombre,telefono,estado
FROM personas
WHERE id='1';
```

B) Serie de tiempo sensor data (clustering y orden)

B1. Crear tabla

```
CREATE TABLE IF NOT EXISTS sensor_data (  
  device_id text,  
  ts timestamp,  
  metric text,  
  value double,  
  PRIMARY KEY ((device_id), ts, metric)  
) WITH CLUSTERING ORDER BY (ts DESC);
```

Verificación (esquema)

columna	tipo	clave
device_id	text	PRIMARY KEY (partition)
ts	timestamp	clustering DESC
metric	text	clustering
value	double	

B2. Inserts de ejemplo

```
INSERT INTO sensor_data (device_id, ts, metric, value)  
VALUES ('dev-01','2026-06-19T08:00:00.000Z','temp',24.8);
```

```
INSERT INTO sensor_data (device_id, ts, metric, value)  
VALUES ('dev-01','2026-06-19T08:05:00.000Z','hum',55.2);
```

```
INSERT INTO sensor_data (device_id, ts, metric, value)  
VALUES ('dev-02','2026-06-19T08:00:00.000Z','temp',22.0);
```

```
INSERT INTO sensor_data (device_id, ts, metric, value)  
VALUES ('dev-02','2026-06-19T08:05:00.000Z','hum',60.5);
```

B3. Verificar datos insertados

```
SELECT * FROM sensor_data;
```

B4. Consulta correcta por clave de partición

```
SELECT device_id, ts, metric, value  
FROM sensor_data  
WHERE device_id = 'dev-01';
```

B5. Consulta por rango de tiempo

```
SELECT device_id, ts, metric, value  
FROM sensor_data  
WHERE device_id = 'dev-01'  
AND ts >= '2026-06-19T08:00:00.000Z'  
AND ts <= '2026-06-19T08:10:00.000Z';
```

B6. Consulta incorrecta para análisis

```
SELECT * FROM sensor_data  
WHERE metric = 'temp';
```

B7. Insertar más datos para observar el orden

```
INSERT INTO sensor_data (device_id, ts, metric, value)
VALUES ('dev-01','2026-06-19T08:10:00.000Z','temp',25.1);
```

```
INSERT INTO sensor_data (device_id, ts, metric, value)
VALUES ('dev-01','2026-06-19T08:15:00.000Z','hum',56.0);
```

```
SELECT device_id, ts, metric, value
FROM sensor_data
WHERE device_id = 'dev-01';
```

Salida esperada:

device_id	ts	metric	value
dev-01	2026-06-19 08:15:00	hum	56.0
dev-01	2026-06-19 08:10:00	temp	25.1
dev-01	2026-06-19 08:05:00	hum	55.2
dev-01	2026-06-19 08:00:00	temp	24.8

Preguntas de cierre

1. ¿Cuál es la clave de partición?
2. ¿Cuáles son las columnas clustering?
3. ¿Para qué sirve CLUSTERING ORDER BY?
4. ¿Por qué no se recomienda consultar solo por metric?
5. ¿Qué tipo de caso real representa esta tabla?

C) Colecciones en Cassandra: set, list y map

Objetivo: Comprender el uso de colecciones en Cassandra para almacenar múltiples valores dentro de una misma fila, diferenciando cuándo usar set, list y map.

C1. Crear tabla user_profile

```
CREATE TABLE IF NOT EXISTS user_profile (
  user_id text,
  intereses set<text>,
  telefonos list<text>,
  redes map<text,text>,
  PRIMARY KEY (user_id)
);
```

Explicación

Esta tabla representa el perfil de un usuario. Cada usuario puede tener varios intereses, varios teléfonos y distintas redes sociales.

Campo	Tipo	Descripción
user_id	text	Identificador único del usuario
intereses	set<text>	Lista de intereses únicos
telefonos	list<text>	Lista ordenada de teléfonos

redes	map<text,text>	Redes sociales en formato clave-valor
-------	----------------	---------------------------------------

C2. Insertar datos iniciales

```
INSERT INTO user_profile (user_id, intereses, telefonos, redes)
VALUES ('u1', {'ia','datos'}, ['+51-999111222'], {'linkedin':'/in/u1'});
```

```
INSERT INTO user_profile (user_id, intereses, telefonos, redes)
VALUES ('u2', {'cloud','backend'}, ['+51-988777666'], {'github':'/u2'});
```

Verificación

```
SELECT * FROM user_profile;
```

Salida esperada

user_id	intereses	telefonos	redes
u1	{'datos','ia'}	['+51-999111222']	{'linkedin':'/in/u1'}
u2	{'backend','cloud'}	['+51-988777666']	{'github':'/u2'}

C3. Diferencia entre set, list y map

Colección	Funcionalidad	¿Permite duplicados?	Caso de uso
set	Guarda valores únicos	No	Intereses, etiquetas, habilidades
list	Guarda valores ordenados	Sí	Teléfonos, historial, pasos
map	Guarda pares clave-valor	No duplica claves	Redes sociales, configuraciones

C4. Actualizar colecciones

Agregar un nuevo interés al set

```
UPDATE user_profile
SET intereses = intereses + {'cloud'}
WHERE user_id='u1';
```

Agregar un nuevo teléfono a la list

```
UPDATE user_profile
SET telefonos = telefonos + ['+51-988777666']
WHERE user_id='u1';
```

Agregar una red social al map

```
UPDATE user_profile
SET redes['github']='rubencarty'
WHERE user_id='u1';
```

Verificación

```
SELECT * FROM user_profile WHERE user_id='u1';
```

Salida esperada

user_id	intereses	telefonos	redes
u1	{'cloud','datos','ia'}	['+51-999111222','+51-988777666']	{'github':'/rubencarty','linkedin':'/in/u1'}

C5. Prueba de duplicados en set

Ejecute nuevamente el siguiente comando:

```
UPDATE user_profile  
SET intereses = intereses + {'ia'}  
WHERE user_id='u1';
```

Luego consulte:

```
SELECT user_id, intereses  
FROM user_profile  
WHERE user_id='u1';
```

Pregunta

¿Por qué no se duplica el valor ia?

C6. Prueba de duplicados en list

Ejecute el siguiente comando:

```
UPDATE user_profile  
SET telefonos = telefonos + ['+51-999111222']  
WHERE user_id='u1';
```

Luego consulte:

```
SELECT user_id, telefonos  
FROM user_profile  
WHERE user_id='u1';
```

Pregunta

¿El teléfono se duplicó?

C7. Prueba de actualización en map

Ejecute el siguiente comando:

```
UPDATE user_profile  
SET redes['github'] = '/nuevo-github'  
WHERE user_id='u1';
```

Luego consulte:

```
SELECT user_id, redes
FROM user_profile
WHERE user_id='u1';
```

Pregunta

¿Se creó otra clave github o se actualizó la existente?

C8. Eliminar elementos de colecciones

Eliminar un interés del set

```
UPDATE user_profile
SET intereses = intereses - {'datos'}
WHERE user_id='u1';
```

Eliminar un teléfono de la list

```
UPDATE user_profile
SET telefonos = telefonos - ['+51-988777666']
WHERE user_id='u1';
```

Eliminar una red social del map

```
DELETE redes['linkedin']
FROM user_profile
WHERE user_id='u1';
```

Verificación final

```
SELECT * FROM user_profile WHERE user_id='u1';
```

Resultado esperado aproximado

user_id	intereses	telefonos	redes
u1	{'cloud','ia'}	['+51-999111222', '+51-999111222']	{'github': '/nuevo-github'}

C9. Análisis del estudiante

Responda brevemente:

1. ¿Qué colección evita duplicados?
2. ¿Qué colección permite datos repetidos?
3. ¿Qué colección trabaja con clave y valor?
4. ¿Cuál usaría para guardar habilidades de un usuario?
5. ¿Cuál usaría para guardar teléfonos?
6. ¿Cuál usaría para guardar redes sociales?

D) LWT: Transacciones ligeras en Cassandra

Objetivo: Comprender el uso de operaciones condicionales en Cassandra mediante IF NOT EXISTS e IF condición.

Las LWT permiten ejecutar cambios solo si se cumple una condición. Son útiles cuando se desea evitar duplicados o controlar actualizaciones críticas.

D1. Insertar solo si no existe

```
INSERT INTO personas (id,nombre,edad,ciudad,email)
VALUES ('2','Ana',30,'Cusco','ana@demo.com') IF NOT EXISTS;
```

Salida esperada

applied	id	nombre	edad	ciudad	email
false	2	Ana	30	Cusco	ana@demo.com

Explicación

El resultado muestra [applied] = false porque ya existe un registro con id='2'. Cassandra no inserta nuevamente el registro.

D2. Insertar un nuevo registro con IF NOT EXISTS

```
INSERT INTO personas (id,nombre,edad,ciudad,email)
VALUES ('10','Rosa',24,'Lima','rosa@demo.com') IF NOT EXISTS;
```

Verificación

```
SELECT * FROM personas WHERE id='10';
```

Salida esperada

applied
true

Explicación

El resultado es true porque no existía una persona con id='10'. Cassandra permite insertar el nuevo registro.

D3. Actualización condicional verdadera

```
UPDATE personas
SET ciudad='Lima'
WHERE id='2'
IF ciudad='Cusco';
```

Salida esperada

applied	ciudad
---------	--------

true	Cusco
------	-------

Verificación

```
SELECT id,nombre,ciudad
FROM personas
WHERE id='2';
```

Resultado esperado

id	nombre	ciudad
2	Ana	Lima

Explicación

La actualización se aplicó porque la condición era verdadera. Ana tenía como ciudad Cusco, por eso Cassandra permitió cambiarla a Lima.

D4. Actualización condicional falsa

Ahora ejecute:

```
UPDATE personas
SET ciudad='Trujillo'
WHERE id='2'
IF ciudad='Cusco';
```

Salida esperada

applied	ciudad
false	Lima

Explicación

La actualización no se aplicó porque Ana ya no tiene ciudad Cusco. Ahora su ciudad es Lima, por eso la condición es falsa.

D5. Verificación final

```
SELECT id,nombre,edad,ciudad,email
FROM personas
WHERE id='2';
```

Resultado esperado

id	nombre	edad	ciudad	email
2	Ana	30	Lima	ana@demo.com

D6. Caso práctico de análisis

Suponga que una aplicación quiere registrar usuarios nuevos, pero no desea duplicar el id.

Comando recomendado

```
INSERT INTO personas (id,nombre,edad,ciudad,email)
VALUES ('11','Marco',31,'Piura','marco@demo.com') IF NOT EXISTS;
```

Pregunta

¿Por qué se usa IF NOT EXISTS?

Respuesta esperada

Se usa para evitar insertar un registro si ya existe una fila con la misma clave primaria. Esto ayuda a controlar duplicados en operaciones importantes.

D7. Preguntas de cierre

1. ¿Qué significa [applied] = true?
2. ¿Qué significa [applied] = false?
3. ¿Para qué sirve IF NOT EXISTS?
4. ¿Para qué sirve IF ciudad='Cusco'?
5. ¿En qué casos reales usaría LWT?

E) Índices secundarios, tabla por consulta y consistencia en Cassandra

Objetivo: Comprender la diferencia entre:

1. **Tabla por consulta:** estrategia recomendada en Cassandra.
2. **Índices secundarios SAI:** opción útil para búsquedas específicas.
3. **Niveles de consistencia:** LOCAL_ONE, LOCAL_QUORUM y ALL.

Parte 1: Consulta mediante tabla por consulta

¿Qué significa tabla por consulta?

Significa crear una tabla específica para responder una consulta frecuente.

E1. Usar el keyspace

USE iot;

E2. Crear tabla personas_by_ciudad

```
CREATE TABLE IF NOT EXISTS personas_by_ciudad (
  ciudad text,
  id text,
  nombre text,
  edad int,
  email text,
  telefono text,
  PRIMARY KEY ((ciudad), id)
);
```

Explicación de la clave primaria

PRIMARY KEY ((ciudad), id)

Elemento	Función
ciudad	Clave de partición

id	Clave clustering
----	------------------

Esto permite consultar eficientemente por ciudad.

E3. Insertar datos en la tabla por consulta

```
INSERT INTO personas_by_ciudad (ciudad,id,nombre,edad,email,telefono)
VALUES ('Lima','1','Juan',23,'juan@demo.com','+51-999111222');
```

```
INSERT INTO personas_by_ciudad (ciudad,id,nombre,edad,email,telefono)
VALUES ('Cusco','2','Ana',30,'ana@demo.com','+51-999222333');
```

```
INSERT INTO personas_by_ciudad (ciudad,id,nombre,edad,email,telefono)
VALUES ('Arequipa','3','Luisa',28,'luisa@demo.com','+51-999333444');
```

```
INSERT INTO personas_by_ciudad (ciudad,id,nombre,edad,email,telefono)
VALUES ('Lima','4','Pedro',35,'pedro@demo.com','+51-999444555');
```

```
INSERT INTO personas_by_ciudad (ciudad,id,nombre,edad,email,telefono)
VALUES ('Lima','5','Carlos',29,'carlos@demo.com','+51-900333444');
```

E4. Verificar datos insertados

```
SELECT * FROM personas_by_ciudad;
```

E5. Consulta recomendada por ciudad

```
SELECT id, nombre, edad, email, telefono
FROM personas_by_ciudad
WHERE ciudad='Lima';
```

Salida esperada

id	nombre	edad	email	telefono
1	Juan	23	juan@demo.com	+51-999111222
4	Pedro	35	pedro@demo.com	+51-999444555
5	Carlos	29	carlos@demo.com	+51-900333444

Pregunta de análisis

¿Por qué esta consulta es eficiente?

Parte 2: Consistencia en Cassandra

¿Qué es la consistencia?

La consistencia define cuántas réplicas deben responder para que una operación sea considerada correcta.

Nivel	Significado	Uso recomendado en laboratorio
LOCAL ONE	Responde una réplica local	Lecturas rápidas

LOCAL_QUORUM	Responde la mayoría de réplicas locales	Escrituras y lecturas más seguras
ALL	Responden todas las réplicas	Máxima consistencia, puede fallar si una réplica no responde

E6. Probar lectura con LOCAL_ONE

CONSISTENCY LOCAL_ONE;

Luego ejecutar:

```
SELECT id, nombre
FROM personas_by_ciudad
WHERE ciudad='Lima';
```

Explicación

LOCAL_ONE pide respuesta a una sola réplica local. Es rápido, pero puede no ser el nivel más fuerte de consistencia.

E7. Probar lectura con LOCAL_QUORUM

CONSISTENCY LOCAL_QUORUM;

Luego ejecutar:

```
SELECT id, nombre
FROM personas_by_ciudad
WHERE ciudad='Lima';
```

Explicación

LOCAL_QUORUM pide respuesta a la mayoría de réplicas locales. Es más seguro que LOCAL_ONE, pero puede ser un poco más costoso.

E8. Probar lectura con ALL

CONSISTENCY ALL;

Luego ejecutar:

```
SELECT id, nombre
FROM personas_by_ciudad
WHERE ciudad='Lima';
```

Posible resultado

La consulta puede funcionar o puede mostrar error si no todas las réplicas responden.

Explicación

ALL exige que todas las réplicas respondan. Es el nivel más estricto, pero también el más sensible a fallos.

E9. Dejar la consistencia recomendada para escrituras

CONSISTENCY LOCAL_QUORUM;

Luego insertar una nueva fila:

```
INSERT INTO personas_by_ciudad (ciudad,id,nombre,edad,email,telefono)
VALUES ('Arequipa','6','María',27,'maria@demo.com','+51-900555666');
```

Verificar:

```
SELECT id, nombre, edad
FROM personas_by_ciudad
WHERE ciudad='Arequipa';
```

Parte 3: Índices secundarios SAI

¿Qué es un índice secundario?

Un índice secundario permite consultar por una columna que no forma parte de la clave primaria.

Ejemplo:

```
WHERE ciudad = 'Arequipa'
```

sobre la tabla personas.

Sin índice, Cassandra normalmente no permite esa consulta de forma directa.

E10. Verificar que existe la tabla personas

DESCRIBE TABLE personas;

Si la tabla no existe, crearla:

```
CREATE TABLE IF NOT EXISTS personas (
  id text,
  nombre text,
  edad int,
  ciudad text,
  email text,
  telefono text,
  PRIMARY KEY (id)
);
```

E11. Insertar datos de prueba en personas

```
INSERT INTO personas (id,nombre,edad,ciudad,email,telefono)
VALUES ('10','Beto',25,'Lima','beto@demo.com','+51-900111000');
```

```
INSERT INTO personas (id,nombre,edad,ciudad,email,telefono)
VALUES ('11','Carla',32,'Arequipa','carla@demo.com','+51-900222000');
```

```
INSERT INTO personas (id,nombre,edad,ciudad,email,telefono)
VALUES ('12','Diego',28,'Cusco','diego@demo.com','+51-900333000');
```

```
INSERT INTO personas (id,nombre,edad,ciudad,email,telefono)
VALUES ('13','Elena',26,'Arequipa','elena@demo.com','+51-900444000');
```

```
INSERT INTO personas (id,nombre,edad,ciudad,email,telefono)
VALUES ('14','Marco',30,'Lima','marco@demo.com','+51-900555000');
```

Verificar:

```
SELECT * FROM personas;
```

E12. Intentar consultar sin índice

```
SELECT id, nombre, ciudad
FROM personas
WHERE ciudad='Arequipa';
```

Pregunta

¿Por qué falla esta consulta?

Parte 4: Crear índices secundarios SAI

E13. Configurar consistencia para crear índices

```
CONSISTENCY LOCAL_QUORUM;
```

E14. Crear índice sobre ciudad

```
DROP INDEX IF EXISTS personas_ciudad_idx;

CREATE INDEX IF NOT EXISTS personas_ciudad_idx
ON personas (ciudad);
```

E15. Crear índice sobre edad

```
DROP INDEX IF EXISTS personas_edad_idx;

CREATE INDEX IF NOT EXISTS personas_edad_idx
ON personas (edad);
```

E16. Verificar índices creados

```
SELECT index_name, kind, options
FROM system_schema.indexes
WHERE keyspace_name='iot'
AND table_name='personas';
```

Salida esperada

index_name	kind	options
personas_ciudad_idx	CUSTOM	...

personas edad idx	CUSTOM	...
-------------------	--------	-----

Nota: en Astra DB puede aparecer como StorageAttachedIndex o SAI dentro de las opciones del índice.

Parte 5: Consultas usando índices

E17. Consultar por ciudad usando índice

```
CONSISTENCY LOCAL_ONE;
SELECT id, nombre, ciudad
FROM personas
WHERE ciudad='Arequipa';
```

Salida esperada

```
id nombre ciudad
11 Carla   Arequipa
13 Elena   Arequipa
```

E18. Consultar por rango de edad usando índice

```
SELECT id, nombre, edad
FROM personas
WHERE edad >= 25 AND edad < 31;
```

Salida esperada

id	nombre	edad
10	Beto	25
12	Diego	28
13	Elena	26
14	Marco	30

E19. Si aparece error de consistencia

Si aparece un error como:

received 0 responses

o

not enough replicas available

Cambiar la consistencia a:

```
CONSISTENCY LOCAL_QUORUM;
```

Y volver a ejecutar:

```
SELECT id, nombre, ciudad
FROM personas
WHERE ciudad='Arequipa';
```

Parte 6: Comparación final

Tabla por consulta

```
SELECT id, nombre
FROM personas_by_ciudad
WHERE ciudad='Arequipa';
```

Índice secundario

```
SELECT id, nombre
FROM personas
WHERE ciudad='Arequipa';
```

Comparación

Criterio	Tabla por consulta	Índice secundario
Rendimiento	Alto	Medio
Escalabilidad	Mejor para grandes volúmenes	Puede degradarse
Diseño recomendado	Sí	Solo para casos puntuales
Consulta por clave primaria	Sí	No necesariamente
Uso en producción	Recomendado	Con cuidado

Parte 7: Preguntas de cierre

1. ¿Qué es una tabla por consulta?
2. ¿Por qué personas_by_ciudad es más eficiente para buscar por ciudad?
3. ¿Para qué sirve un índice secundario?
4. ¿Qué diferencia hay entre LOCAL_ONE, LOCAL_QUORUM y ALL?
5. ¿Cuál estrategia es más recomendable en Cassandra para sistemas grandes?
6. ¿Por qué no se debe abusar de los índices secundarios?

Parte Final del Laboratorio — Cassandra Aplicado

Objetivo general

Aplicar los conceptos trabajados en el laboratorio:

- Modelado por consulta.
- Dual Write.
- Series temporales.
- Columnas clustering.
- TTL.
- LWT.
- Contadores distribuidos.
- Datos tipo sensor e IoT.

Parte 1. Sincronización de tablas mediante Dual Write

Contexto

En Cassandra es común duplicar información en varias tablas para responder consultas diferentes.

En este laboratorio se mantendrán sincronizadas dos tablas:

- personas
- personas_by_ciudad

La tabla personas permite consultar por id.

La tabla personas_by_ciudad permite consultar por ciudad.

Paso 1. Verificar tabla personas

DESCRIBE TABLE personas;

Si no existe, crearla:

```
CREATE TABLE IF NOT EXISTS personas (  
  id text,  
  nombre text,  
  edad int,  
  ciudad text,  
  email text,  
  telefono text,  
  PRIMARY KEY (id)  
);
```

Paso 2. Verificar tabla personas_by_ciudad

DESCRIBE TABLE personas_by_ciudad;

Si no existe, crearla:

```
CREATE TABLE IF NOT EXISTS personas_by_ciudad (  
  ciudad text,  
  id text,  
  nombre text,  
  edad int,  
  email text,  
  telefono text,  
  PRIMARY KEY ((ciudad), id)  
);
```

Paso 3. Registrar una nueva persona en ambas tablas

BEGIN BATCH

```
INSERT INTO personas  
(id,nombre,edad,ciudad,email,telefono)
```

```
VALUES
('13','Maria',27,'Lima','maria@demo.com','+51-900555666');
```

```
INSERT INTO personas_by_ciudad
(ciudad,id,nombre,edad,email,telefono)
VALUES
('Lima','13','Maria',27,'maria@demo.com','+51-900555666');
```

```
APPLY BATCH;
```

Paso 4. Verificar en tabla principal

```
SELECT id,nombre,ciudad,email,telefono
FROM personas
WHERE id='13';
```

Resultado esperado

id	nombre	ciudad	email	telefono
13	Maria	Lima	maria@demo.com	+51-900555666

Paso 5. Verificar en tabla por ciudad

```
SELECT id,nombre,edad,email,telefono
FROM personas_by_ciudad
WHERE ciudad='Lima';
```

Paso 6. Cambio de ciudad

Suponga que María se muda de Lima a Cusco.

En Cassandra no basta con actualizar la tabla personas. También se debe actualizar la tabla diseñada para buscar por ciudad.

```
BEGIN BATCH
```

```
UPDATE personas
SET ciudad='Cusco'
WHERE id='13';
```

```
DELETE FROM personas_by_ciudad
WHERE ciudad='Lima'
AND id='13';
```

```
INSERT INTO personas_by_ciudad
(ciudad,id,nombre,edad,email,telefono)
VALUES
('Cusco','13','Maria',27,'maria@demo.com','+51-900555666');
```

```
APPLY BATCH;
```

Paso 7. Verificar que ya no está en Lima

```
SELECT id,nombre
FROM personas_by_ciudad
WHERE ciudad='Lima';
```

María ya no debe aparecer en Lima.

Paso 8. Verificar que ahora está en Cusco

```
SELECT id,nombre,edad,email,telefono
```

```
FROM personas_by_ciudad
```

```
WHERE ciudad='Cusco';
```

Resultado esperado

id	nombre	edad	email	telefono
13	Maria	27	maria@demo.com	+51-900555666

Pregunta de reflexión

¿Por qué Cassandra requiere actualizar ambas tablas cuando se modifica la ciudad de una persona?

Parte 2. Modelado de series temporales para sensores IoT

Contexto

Cassandra es muy utilizada para almacenar datos de:

- Sensores IoT.
- Telemetría.
- Monitoreo ambiental.
- Sistemas industriales.
- Aplicaciones financieras.
- Métricas de servidores.

En este caso se registrarán lecturas de sensores por dispositivo, métrica y tiempo.

Paso 1. Crear tabla optimizada por dispositivo y métrica

```
CREATE TABLE IF NOT EXISTS sensor_by_metric (  
  device_id text,  
  metric text,  
  ts timestamp,  
  value double,  
  PRIMARY KEY ((device_id, metric), ts)  
)  
WITH CLUSTERING ORDER BY (ts DESC);
```

Explicación de la clave primaria

PRIMARY KEY ((device_id, metric), ts)

Elemento	Tipo	Función
device_id	Partition key	Identifica el dispositivo
metric	Partition key	Identifica la métrica
ts	Clustering key	Ordena las lecturas por tiempo
value	Columna normal	Valor medido

Concepto clave: columna ancha

En Cassandra, una misma partición puede almacenar muchas filas ordenadas por columnas clustering.

Ejemplo conceptual:

Partición	Filas internas ordenadas
dev-01 + temp	08:25, 08:20, 08:15, 08:10, 08:05
dev-01 + hum	08:25, 08:20, 08:15, 08:10, 08:05
dev-02 + temp	08:25, 08:20, 08:15, 08:10, 08:05

Esto se conoce como **wide-column model** o modelo de columna ancha.

Paso 2. Insertar varias lecturas de temperatura

```
INSERT INTO sensor_by_metric (device_id,metric,ts,value)  
VALUES ('dev-01','temp','2026-06-19T08:00:00.000Z',24.8);
```

```
INSERT INTO sensor_by_metric (device_id,metric,ts,value)
VALUES ('dev-01','temp','2026-06-19T08:05:00.000Z',25.0);
```

```
INSERT INTO sensor_by_metric (device_id,metric,ts,value)
VALUES ('dev-01','temp','2026-06-19T08:10:00.000Z',25.2);
```

```
INSERT INTO sensor_by_metric (device_id,metric,ts,value)
VALUES ('dev-01','temp','2026-06-19T08:15:00.000Z',25.5);
```

```
INSERT INTO sensor_by_metric (device_id,metric,ts,value)
VALUES ('dev-01','temp','2026-06-19T08:20:00.000Z',25.7);
```

Paso 3. Insertar lecturas de humedad

```
INSERT INTO sensor_by_metric (device_id,metric,ts,value)
VALUES ('dev-01','hum','2026-06-19T08:00:00.000Z',55.2);
```

```
INSERT INTO sensor_by_metric (device_id,metric,ts,value)
VALUES ('dev-01','hum','2026-06-19T08:05:00.000Z',56.0);
```

```
INSERT INTO sensor_by_metric (device_id,metric,ts,value)
VALUES ('dev-01','hum','2026-06-19T08:10:00.000Z',56.5);
```

```
INSERT INTO sensor_by_metric (device_id,metric,ts,value)
VALUES ('dev-01','hum','2026-06-19T08:15:00.000Z',57.1);
```

```
INSERT INTO sensor_by_metric (device_id,metric,ts,value)
VALUES ('dev-01','hum','2026-06-19T08:20:00.000Z',57.8);
```

Paso 4. Insertar datos de otro dispositivo

```
INSERT INTO sensor_by_metric (device_id,metric,ts,value)
VALUES ('dev-02','temp','2026-06-19T08:00:00.000Z',22.0);
```

```
INSERT INTO sensor_by_metric (device_id,metric,ts,value)
VALUES ('dev-02','temp','2026-06-19T08:05:00.000Z',22.4);
```

```
INSERT INTO sensor_by_metric (device_id,metric,ts,value)
VALUES ('dev-02','temp','2026-06-19T08:10:00.000Z',22.9);
```

```
INSERT INTO sensor_by_metric (device_id,metric,ts,value)
VALUES ('dev-02','temp','2026-06-19T08:15:00.000Z',23.1);
```

```
INSERT INTO sensor_by_metric (device_id,metric,ts,value)
VALUES ('dev-02','temp','2026-06-19T08:20:00.000Z',23.5);
```

Paso 5. Consultar las últimas mediciones de temperatura

```
SELECT ts,value
FROM sensor_by_metric
WHERE device_id='dev-01'
AND metric='temp'
LIMIT 5;
```

Resultado esperado

ts	value
2026-06-19 08:20:00	25.7
2026-06-19 08:15:00	25.5
2026-06-19 08:10:00	25.2
2026-06-19 08:05:00	25.0
2026-06-19 08:00:00	24.8

Paso 6. Consultar por rango de tiempo

```
SELECT ts,value
FROM sensor_by_metric
WHERE device_id='dev-01'
AND metric='temp'
AND ts >= '2026-06-19T08:05:00.000Z'
AND ts <= '2026-06-19T08:15:00.000Z';
```

Resultado esperado

ts	value
2026-06-19 08:15:00	25.5
2026-06-19 08:10:00	25.2
2026-06-19 08:05:00	25.0

Paso 7. Consulta incorrecta para análisis

Ejecute:

```
SELECT ts,value
FROM sensor_by_metric
WHERE metric='temp';
```

Resultado esperado

La consulta puede fallar porque no se está usando la clave de partición completa.

Explicación

La clave de partición está compuesta por:

(device_id, metric)

Por eso Cassandra necesita ambos valores para ubicar correctamente la partición.

1. ¿Cuál es la clave de partición?
2. ¿Cuál es la columna clustering?
3. ¿Por qué las lecturas aparecen de la más reciente a la más antigua?
4. ¿Qué representa una columna ancha en este ejemplo?
5. ¿Por qué no se puede consultar solo por metric?

Parte 3. TTL: expiración automática de datos

Muchos sistemas almacenan información temporal:

- Logs.
- Sesiones.
- Eventos.
- Telemetría temporal.
- Tokens.
- Alertas.

Cassandra permite definir un tiempo de vida mediante TTL.

Paso 1. Crear tabla eventos_ttl

```
CREATE TABLE IF NOT EXISTS eventos_ttl (
  id text,
  payload text,
```

```
tipo text,  
created_at timestamp,  
PRIMARY KEY(id)  
);
```

Paso 2. Insertar evento temporal con TTL

```
INSERT INTO eventos_ttl  
(id,payload,tipo,created_at)  
VALUES  
('e1','evento demo','sensor',toTimestamp(now()))  
USING TTL 30;
```

Paso 3. Verificar tiempo restante

```
SELECT id,  
       payload,  
       tipo,  
       TTL(payload) AS segundos_restantes  
FROM eventos_ttl  
WHERE id='e1';
```

Resultado esperado

id	payload	tipo	segundos_restantes
e1	evento demo	sensor	30 aprox.

Paso 4. Ejecutar nuevamente después de unos segundos

```
SELECT id,  
       payload,  
       tipo,  
       TTL(payload) AS segundos_restantes  
FROM eventos_ttl  
WHERE id='e1';  
El valor de segundos_restantes debe disminuir.
```

Paso 5. Esperar 30 segundos y consultar

```
SELECT *  
FROM eventos_ttl  
WHERE id='e1';
```

Resultado esperado

El registro ya no debe aparecer.

Concepto clave

TTL permite implementar políticas de retención sin tareas programadas adicionales.

Parte 4. Transacciones ligeras LWT

Contexto

Las LWT permiten ejecutar operaciones solo si se cumple una condición.

Se usan en casos como:

- Evitar duplicados.
- Registrar usuarios únicos.
- Controlar reservas.
- Cambiar estados de forma segura.
- Validar reglas de negocio.

Paso 1. Crear tabla usuarios

```
CREATE TABLE IF NOT EXISTS usuarios (  
  user_id text,  
  email text,  
  estado text,  
  PRIMARY KEY(user_id)  
);
```

Paso 2. Registrar usuario

```
INSERT INTO usuarios  
(user_id,email,estado)  
VALUES  
(u100,'a@demo.com','activo');
```

Paso 3. Evitar duplicado con IF NOT EXISTS

```
INSERT INTO usuarios  
(user_id,email,estado)  
VALUES  
(u100,'otro@demo.com','activo')  
IF NOT EXISTS;
```

Resultado esperado

applied

false

Explicación

La operación no se ejecuta porque el registro u100 ya existe.

Paso 4. Insertar un usuario nuevo con LWT

```
INSERT INTO usuarios  
(user_id,email,estado)  
VALUES  
(u101,'b@demo.com','activo')  
IF NOT EXISTS;
```

Resultado esperado

applied

true

Paso 5. Actualización condicional

```
UPDATE usuarios  
SET estado='bloqueado'  
WHERE user_id=u101  
IF estado='activo';
```

Resultado esperado

applied estado

true activo

Paso 6. Verificar cambio

```
SELECT *  
FROM usuarios  
WHERE user_id=u101;
```

Resultado esperado

user_id	email	estado
u101	b@demo.com	bloqueado

Paso 7. Intentar cambio con condición falsa

```
UPDATE usuarios
SET estado='eliminado'
WHERE user_id='u101'
IF estado='activo';
```

Resultado esperado

applied	estado
false	bloqueado

Preguntas de refuerzo

1. ¿Qué significa [applied] = true?
2. ¿Qué significa [applied] = false?
3. ¿Para qué sirve IF NOT EXISTS?
4. ¿Cuándo usaría LWT en una aplicación real?

Parte 5. Contadores distribuidos

Contexto

Los contadores se usan para contar eventos distribuidos:

- Visitas a una página.
- Likes.
- Descargas.
- Reproducciones.
- Métricas de uso.
- Rankings.

Importante: una tabla con contador no debe mezclar columnas normales que se actualicen libremente. El contador se incrementa o decrementa mediante UPDATE.

Paso 1. Crear tabla page_views

```
CREATE TABLE IF NOT EXISTS page_views (
  url text,
  day date,
  views counter,
  PRIMARY KEY ((url), day)
);
```

Paso 2. Incrementar visitas

```
UPDATE page_views
SET views = views + 1
WHERE url='https://utp.edu.pe'
AND day=toDate(now());
UPDATE page_views
SET views = views + 5
WHERE url='https://utp.edu.pe'
AND day=toDate(now());
```

Paso 3. Consultar contador

```
SELECT *
FROM page_views
WHERE url='https://utp.edu.pe';
```

Resultado esperado

url	day	views
https://utp.edu.pe	fecha actual	6

Paso 4. Incrementar otro recurso

```
UPDATE page_views  
SET views = views + 3  
WHERE url='https://aulavirtual.utp.edu.pe'  
AND day=toDate(now());
```

Paso 5. Consultar segundo contador

```
SELECT *  
FROM page_views  
WHERE url='https://aulavirtual.utp.edu.pe';
```

Preguntas de refuerzo

1. ¿Para qué sirve un contador?
2. ¿Por qué se usa UPDATE y no INSERT?
3. ¿Qué tipo de sistemas usan contadores?
4. ¿Qué diferencia hay entre un contador y un campo numérico normal?

Parte 6. Mini reto final del laboratorio

La empresa SmartCampus desea monitorear sensores instalados en aulas inteligentes. Cada sensor envía mediciones de temperatura y humedad cada 5 minutos.

El sistema necesita responder estas consultas:

1. Ver las últimas mediciones de temperatura de un dispositivo.
2. Ver las últimas mediciones de humedad de un dispositivo.
3. Consultar mediciones en un rango de tiempo.
4. Eliminar automáticamente eventos temporales.
5. Contar accesos al dashboard de monitoreo.

Actividad del estudiante

Crear o usar las tablas desarrolladas:

- sensor_by_metric
- eventos_ttl
- page_views

Ejecutar consultas y responder:

1. ¿Qué tabla usaría para consultar las últimas temperaturas?
2. ¿Qué columna permite ordenar las mediciones por tiempo?
3. ¿Qué comando permite que un dato expire automáticamente?
4. ¿Qué tabla permite contar visitas?
5. ¿Qué ventaja tiene Cassandra para sensores IoT?

Respuesta guía

1. Usaría sensor_by_metric.
2. La columna ts, porque es clustering key.
3. Se usa USING TTL.
4. La tabla page_views.
5. Cassandra permite almacenar grandes volúmenes de datos distribuidos, consultar por particiones y manejar series temporales de forma eficiente.

Cierre del laboratorio

En este laboratorio se aplicaron los principales conceptos de Cassandra:

- Las tablas se diseñan según las consultas.
- La duplicación de datos es normal en Cassandra.
- Dual Write permite mantener varias tablas sincronizadas.
- Las series temporales se modelan con claves de partición y columnas clustering.
- CLUSTERING ORDER BY permite ordenar los datos dentro de una partición.
- TTL permite eliminar datos automáticamente.
- LWT permite operaciones condicionales.
- Los contadores permiten registrar métricas distribuidas.

La idea más importante es:

En Cassandra no se empieza diseñando relaciones, se empieza diseñando consultas.

Actividad Integradora: Sistema de Monitoreo Inteligente SmartCampus

Diseñar e implementar una solución en Cassandra para almacenar y consultar información de sensores instalados en un campus inteligente.

La solución debe aplicar los conceptos desarrollados durante el laboratorio:

- Modelado orientado a consultas.
- Claves de partición y clustering.
- Tablas por consulta.
- Dual Write.
- Series temporales.
- TTL.
- Colecciones.
- Transacciones ligeras (LWT).
- Contadores distribuidos.
- Consistencia.

Contexto

La empresa SmartCampus implementará sensores IoT en diferentes aulas de la universidad.

Cada sensor enviará información cada 5 minutos sobre:

- Temperatura.
- Humedad.
- Calidad del aire.

Además, el sistema debe registrar información de los dispositivos, eventos temporales, accesos al dashboard y usuarios responsables de los sensores.

El área de TI ha solicitado construir una base de datos Cassandra que permita responder eficientemente las consultas más frecuentes.

Requerimientos

Parte 1. Gestión de dispositivos

Diseña una tabla que permita registrar dispositivos IoT.

Cada dispositivo debe almacenar como mínimo:

- Código del dispositivo.
- Nombre.
- Ubicación.
- Estado.
- Responsable.

Actividades

1. Crear la tabla.
2. Registrar al menos 5 dispositivos.
3. Consultar un dispositivo específico.
4. Actualizar el estado de un dispositivo.
5. Eliminar un dispositivo.

Parte 2. Colecciones

Cada dispositivo puede tener:

- Varias etiquetas.
- Varios números de contacto.
- Varias configuraciones.

Actividades

1. Utilizar una colección set.
2. Utilizar una colección list.
3. Utilizar una colección map.
4. Demostrar qué ocurre cuando se intenta insertar un valor duplicado en el set.
5. Demostrar qué ocurre cuando se inserta un valor duplicado en el list.
6. Actualizar un valor dentro del map.

Parte 3. Series temporales

Diseña una tabla optimizada para almacenar lecturas de sensores.

Cada lectura debe contener:

- Dispositivo.
- Tipo de métrica.
- Fecha y hora.
- Valor.

Actividades

1. Diseñar la clave primaria.
2. Configurar el ordenamiento para que las lecturas más recientes aparezcan primero.
3. Registrar al menos 15 lecturas de temperatura.
4. Registrar al menos 15 lecturas de humedad.
5. Consultar las últimas 5 mediciones.
6. Consultar mediciones dentro de un rango de tiempo.

Parte 4. Tabla por consulta

El área de monitoreo necesita consultar rápidamente todos los sensores de una sede específica.

Actividades

1. Diseñar una segunda tabla optimizada para consultar dispositivos por ubicación.
2. Implementar Dual Write para mantener ambas tablas sincronizadas.
3. Registrar un nuevo dispositivo utilizando Dual Write.
4. Simular el cambio de ubicación de un dispositivo.
5. Verificar que ambas tablas permanezcan consistentes.

Parte 5. TTL

Los eventos de alerta solo deben mantenerse durante un tiempo limitado.

Actividades

1. Crear una tabla para eventos temporales.
2. Registrar al menos tres eventos utilizando TTL.
3. Consultar el tiempo restante mediante la función TTL().
4. Verificar qué ocurre cuando el tiempo expira.

Parte 6. Transacciones ligeras (LWT)

El sistema no debe permitir registrar dos dispositivos con el mismo código.

Actividades

1. Insertar un dispositivo utilizando IF NOT EXISTS.
2. Intentar registrar nuevamente el mismo dispositivo.
3. Analizar el resultado obtenido.
4. Realizar una actualización condicional utilizando IF.

Parte 7. Contadores distribuidos

El dashboard de monitoreo registra cada acceso realizado por los operadores.

Actividades

1. Crear una tabla de contadores.
2. Incrementar varias veces el número de accesos.
3. Consultar el valor acumulado.
4. Registrar accesos para dos dashboards distintos.

Parte 8. Consistencia

Investigue y pruebe los siguientes niveles de consistencia:

- LOCAL_ONE
- LOCAL_QUORUM
- ALL

Actividades

1. Realizar una consulta utilizando cada nivel.
2. Describir la diferencia observada.
3. Indicar cuál utilizaría para:
 - Un sistema IoT de alta velocidad.
 - Un sistema bancario.
 - Un dashboard de monitoreo.

Entregables:

- Capturas de pantalla completas.
- Código CQL ejecutado.
- Resultados obtenidos.

Responder las siguientes preguntas:

1. ¿Por qué Cassandra utiliza modelado orientado a consultas?
2. ¿Qué ventaja tiene una tabla por consulta respecto a un índice secundario?
3. ¿Cuál es la función de una clave de partición?
4. ¿Cuál es la función de una columna clustering?
5. ¿Qué ventajas ofrece Cassandra para aplicaciones IoT?
6. ¿Cuándo utilizaría TTL?
7. ¿Qué problema resuelven las LWT?
8. ¿Para qué sirven los contadores distribuidos?